

Algorithm for file updates in Python

Project description

As a security professional working in a healthcare organisation, I was responsible for maintaining an allow list of IP addresses authorised to access restricted patient information. The allow list was stored in a text file and a separate remove list showed IP addresses that should no longer have access to sensitive resources. To improve efficiency and reduce manual administration, I developed a Python algorithm that automatically reviews the allow list, removes unauthorised IP addresses, and updates the file with the new data. This project demonstrates the use of Python for file handling, data processing, iteration, conditional logic, and access control management.

Open the file that contains the allow list

```
# Assign `import_file` to the name of the file  
import_file = "allow_list.txt"  
  
# Build `with` statement to read in the initial contents of the file  
with open(import_file, "r") as file:
```

The first step of the algorithm was to access the file containing the allow list. I assigned the file name to the variable `import_file`, which makes the code easier to maintain and modify in the future. To open the file, I used a `with` statement together with the `open()` function.

The `with` statement is commonly used when working with files because it automatically manages system resources. Once the code in the `with` block finishes running, the file is automatically closed. This reduces the risk of leaving files open accidentally.

The `open()` function was used with the `"r"` argument, which uses the file in read mode. Since the aim at this point was to retrieve existing data from the allow list, read mode was the right option. The `as` keyword stores the opened file object in the variable `file`, allowing it to be referenced throughout the rest of the block.

Read the file contents

```
with open(import_file, "r") as file:  
    # Use `.read()` to read the imported file and store it in a variable named `ip_addresses`  
    ip_addresses = file.read()
```

After opening the file, I used the `.read()` method to retrieve its contents. The `.read()` method reads all data contained within the file and converts it into a string.

The contents of the file were stored in the variable `ip_addresses`. Storing the information in a variable allows the data to be worked with later in the algorithm. At this point, the IP addresses existed as a single string containing all values from the allow list.

Reading the file contents is necessary because the algorithm must compare current IP addresses against a separate list of addresses that should be removed.

Convert the string into a list

```
# Use `.split()` to convert `ip_addresses` from a string to a list  
ip_addresses = ip_addresses.split()
```

To remove individual IP addresses from the allow list, the string needed to be converted into a list. This was done using the `.split()` method.

The `.split()` method separates a string into multiple elements and stores them in a list. By default, it uses whitespace as a separator. Since each IP address is separated in the file, this method creates a list where each IP address becomes an individual element.

Converting the string into a list makes it easier to process individual IP addresses. Lists give functionality that allows items to be searched, compared, and removed efficiently. This conversion prepares the data for the next part of the algorithm.

Iterate through the remove list

```
# Build iterative statement  
# Name loop variable `element`  
# Loop through `ip_addresses`  
  
for element in ip_addresses:
```

The algorithm must compare every IP address in the remove list against the allow list. To do this, I used a `for` loop.

The `for` loop iterates through each element stored in the `remove_list`. During each iteration, the current value is assigned to the variable `element`.

Using a loop removes the need to manually check each IP address one at a time. This approach allows the same logic to be applied repeatedly until every item in the remove list has been processed.

Iteration is an important programming concept because it allows large collections of data to be handled efficiently and consistently.

Remove IP addresses that are on the remove list

```
for element in ip_addresses:  
  
    # Create conditional statement to evaluate if 'element' is in 'ip_addresses'  
  
    if element in remove_list:  
  
        # use the '.remove()' method to remove  
        # elements from 'ip_addresses'  
  
        ip_addresses.remove(element)
```

In the loop, I used a conditional statement to see whether the current IP address from the remove list exists in the allow list.

The `if` statement checks membership using the `in` operator. If the current value stored in `element` exists in the `ip_addresses` list, the condition becomes true.

When a match is found, the `.remove()` method removes the matching IP address from the allow list. This makes sure that users associated with those IP addresses will no longer have authorisation to access restricted resources.

The conditional check is important because attempting to remove an item that does not exist could create an error. Verifying membership before removal helps make sure the algorithm runs successfully.

Using `.remove()` in this way is possible because there are no duplicate IP addresses in the `ip_addresses` list. Since each IP address appears only once, removing the matching value completely removes its authorisation from the allow list.

Update the file with the revised list of IP addresses

```
# Convert `ip_addresses` back to a string so that it can be written into the text file  
ip_addresses = " ".join(ip_addresses)
```

```
# Build `with` statement to rewrite the original file  
with open(import_file, "w") as file:  
    # Rewrite the file, replacing its contents with `ip_addresses`  
    file.write(ip_addresses)
```

After removing unauthorised IP addresses, the new allow list needed to be written back to the file.

Because the `.write()` method needs a string, I first converted the list back into string format using the `.join()` method. The string `"\n"` was used as a separator so that each IP address would be on its own line within the file.

Once the list was converted back into a string, I used a second `with` statement together with the `open()` function. This time, the file was opened using `"w"` mode, which uses the file in write mode.

The `.write()` method was then used to overwrite the existing contents of the file with the updated allow list. This completed the update process and made sure that only authorised IP addresses stayed in the file.

Summary

In this project, I developed a Python algorithm that automates the process of updating an allow list used to control access to restricted resources. The algorithm starts by opening a file and reading its contents into a string using the `.read()` method. The string is converted into a list using `.split()` so that individual IP addresses can be processed.

A `for` loop iterates through a remove list and compares each value against the allow list. When a matching IP address is found, the `.remove()` method deletes it from the list. After all unauthorised addresses have been removed, the `.join()` method converts the list back into a string.

Finally, the updated data is written back to the file using a second `with` statement and the `.write()` method. This solution improves efficiency, reduces manual effort, and helps maintain secure access control by making sure that only approved IP addresses remain authorised.